

Lecture 4: Relational Algebra (Part 2)

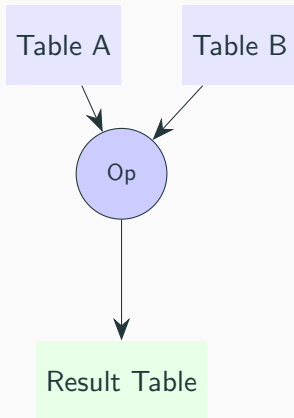
Amit Kumar Dhar

August 6, 2025

Introduction

What is Relational Algebra?

- A procedural query language.
- Forms the theoretical foundation for SQL.
- Takes relations (tables) as input and produces new relations as output.
- Operations are composed to form complex queries.



Our Running Example: A Student Database

Students

sid	sname	Discip
101	Isha	CSE
102	Parth	DSAI
103	Juhi	CSE
104	Suresh	MA

Enrolled

sid	cid	grade
101	CSL303	A
102	EEL201	B
103	CSL303	B
101	CSL211	A

Fundamental Operations

Selection (σ)

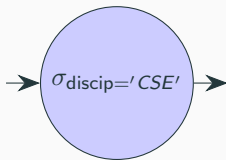
Concept: Filters rows based on a predicate (condition).

Example: Find all students in the 'CSE' discip.

Formula:

$$\sigma_{\text{discip}='CSE'}(\text{Students})$$

sid	sname	discip
101	Alice	CSE
102	Parth	DSAI
103	Juhi	CSE
104	Suresh	MA



sid	sname	discip
101	Isha	CSE
103	Juhi	CSE

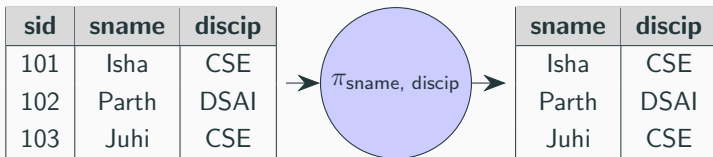
Projection (π)

Concept: Selects a subset of columns, removing duplicates.

Example: Get the names and discips of all students.

Formula:

$$\pi_{\text{sname, discip}}(\text{Students})$$

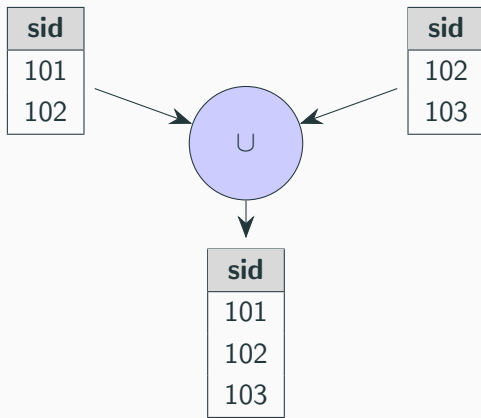


Union ($\cup$)

Concept: Combines rows from two union-compatible tables. Tables must have the same number of columns and corresponding columns must have the same domain.

$\pi_{sid}(\sigma_{discip='CSE'}(Students))$

$\pi_{sid}(\sigma_{discip \neq 'CSE'}(Students))$



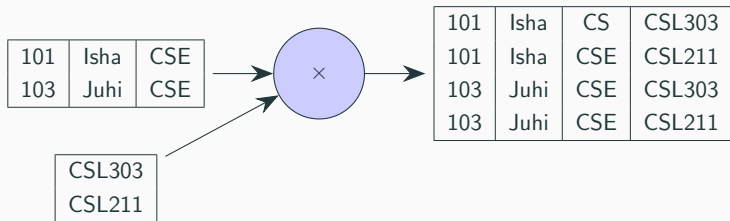
Cartesian Product ($\times$)

Concept: Combines every row from the first table with every row from the second table.

Formula:

$$\sigma_{\text{discip}='CSE'}(\text{Students}) \times \pi_{\text{cid}}(\text{Enrolled})$$

(Example uses subsets for brevity)



The Join Family

Join ($\bowtie$): The Core Idea

Joins are the most common way to combine information from multiple related tables.

The fundamental join is a **Cartesian Product followed by a Selection**.

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

- R and S are relations.
- θ is the join condition (a predicate).

We will explore several variations of this powerful operator.

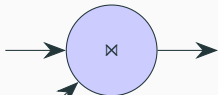
Natural Join ($\bowtie$)

Concept: The most common join. It automatically combines tables based on columns with the **same name**.

1. Compute Cartesian Product.
2. Select rows where values in common columns are equal.
3. Project away the duplicate common columns.

Students $\bowtie$ Enrolled

sid	sname
101	Isha
102	Parth
103	Juhi



sid	sname	cid
101	Isha	CSL303
101	Isha	CSL211
102	Parth	EEL201

sid	cid
101	CSL303
102	EEL201
101	CSL211

Theta Join ($\bowtie_{\theta}$) and Equi-Join

- **Theta Join:** A generalized join where the condition θ can be any comparison ($<, >, =, \neq, \leq, \geq$).

$$R \bowtie_{\theta} S$$

- **Equi-Join:** A specific type of Theta Join where the condition θ **only uses equality (=)**. The result contains redundant columns.

Example (Equi-Join):

Students $\bowtie_{\text{Students.sid}=\text{Enrolled.sid}}$ Enrolled

Students.sid	sname	discip	Enrolled.sid	cid	grade
101	Isha	CSE	101	CSL303	A
101	Isha	CSE	101	CSL211	A
102	Parth	DSAI	102	EEL201	B
103	Juhi	CSE	103	CSL303	B

The Problem with Inner Joins

What happens to students who haven't enrolled in any courses?
Or courses with no students?

Inner Joins (Natural, Theta, Equi) only keep rows that have a match in the other table.

In our example, student 'Suresh (104)' is not in the 'Enrolled' table.

Students ⋈ Enrolled

...Suresh disappears from the result!

Solution: Outer Joins.

Left Outer Join (⋈)

Concept: Keeps all rows from the **left** table. If a row has no match in the right table, the right table's columns are filled with NULL.

Formula:

Students ⋈ Enrolled

Result of Left Outer Join

sid	sname	discip	cid	grade
101	Isha	CSE	CSL303	A
101	Isha	CSE	CSL211	A
102	Parth	DSAI	EEL201	B
103	Juhi	CSE	CSL303	B
104	Suresh	MA	NULL	NULL

Right Outer Join (⋈_R)

Concept: The opposite of Left Outer Join. Keeps all rows from the **right** table. Unmatched rows have NULLs for the left table's columns.

Formula:

Students ⋈_R Enrolled

(Imagine a course 'CSL101' in 'Enrolled' that no student is taking)

Result of Right Outer Join

sid	sname	discip	cid	grade
101	Isha	CSE	CSL303	A
101	Isha	CSE	CSL211	A
102	Parth	DSAI	EEL201	B
103	Juhi	CSE	CSL303	B
NULL	NULL	NULL	CSL101	C

Full Outer Join ($\bowtie$)

Concept: Keeps all rows from **both** tables. Fills in NULLs on either side as needed. It's the union of the Left and Right Outer Join results.

Formula:

Students $\bowtie$ Enrolled

Result of Full Outer Join

sid	sname	discip	cid	grade
101	Isha	CSE	CSL303	A
101	Isha	CSE	CSL211	A
102	Parth	DSAI	EEL201	B
103	Juhi	CSE	CSL303	B
104	Suresh	MA	NULL	NULL
NULL	NULL	NULL	CSL101	C

Complex Queries

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

$$\text{A_Grades} \leftarrow \sigma_{\text{grade}='A'}(\text{Enrolled})$$

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

$$\text{A_Grades} \leftarrow \sigma_{\text{grade}='A'}(\text{Enrolled})$$

Step 3: Link students to their 'A' grades.

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

$$\text{A_Grades} \leftarrow \sigma_{\text{grade}='A'}(\text{Enrolled})$$

Step 3: Link students to their 'A' grades.

$$\text{Result} \leftarrow \text{CSE_Students} \bowtie \text{A_Grades}$$

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

$$\text{A_Grades} \leftarrow \sigma_{\text{grade}='A'}(\text{Enrolled})$$

Step 3: Link students to their 'A' grades.

$$\text{Result} \leftarrow \text{CSE_Students} \bowtie \text{A_Grades}$$

Step 4: Project just the student names.

Putting It All Together

Query: Find the names of all 'CSE' students who got an 'A' in a course.

Step 1: Find all CSE students.

$$\text{CSE_Students} \leftarrow \sigma_{\text{discip}='CSE'}(\text{Students})$$

Step 2: Find all enrollments with an 'A' grade.

$$\text{A_Grades} \leftarrow \sigma_{\text{grade}='A'}(\text{Enrolled})$$

Step 3: Link students to their 'A' grades.

$$\text{Result} \leftarrow \text{CSE_Students} \bowtie \text{A_Grades}$$

Step 4: Project just the student names.

$$\pi_{\text{name}}(\text{Result})$$

Putting It All Together

Final Combined Formula:

$$\pi_{\text{sname}}(\sigma_{\text{discip}='CSE'}(\text{Students}) \bowtie \sigma_{\text{grade}='A'}(\text{Enrolled}))$$

Result:

sname
Isha

Thank You!
Questions?